

Trabajo Practico Integrador

Solución Chat Bot Viáticos

Alumnos - Grupo n° XX
Ernesto Abel Fatale - Comisión 20
Juan Amador Lizarrondo Buffa - Comisión 19

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.

Organización Empresarial

Docente Titular
Prof. Gabriela Martínez
Docente Tutor
Prof. Francisco Varberde.

18 de Junio de 2026

Tabla de contenido

Tabla de contenido	2
Trabajo Práctico N°01	3
INTRODUCCIÓN	3
OBJETIVOS	3
CONSIGNA	4
ARQUITECTURA TECNICA DE LA SOLUCION	6
REGLAS DE NEGOCIO	6
DICCIONARIO DE DATOS	6
MANUAL DE USUARIO	7
PRUEBAS Y MANEJO DE ERRORES	7
USO DE HERRAMIENTAS DE IA	8
REPOSITORIO	9
CONCLUSIÓN	10

Trabajo Práctico N°01

INTRODUCCIÓN

Para un futuro Técnico Universitario en Programación (TUP) este trabajo integrador representa el puente crítico entre el conocimiento académico y la práctica profesional. En el entorno laboral actual, la programación no es una actividad aislada; su valor real reside en su capacidad para resolver problemas específicos y optimizar procesos de negocio.

Este proyecto los desafía a dejar de ser solo redactores de código para convertirse en consultores tecnológicos capaces de identificar inefficiencias y proponer soluciones automatizadas.

No se evaluará la complejidad tecnológica sino la coherencia entre el modelo BPMN y la lógica implementada.

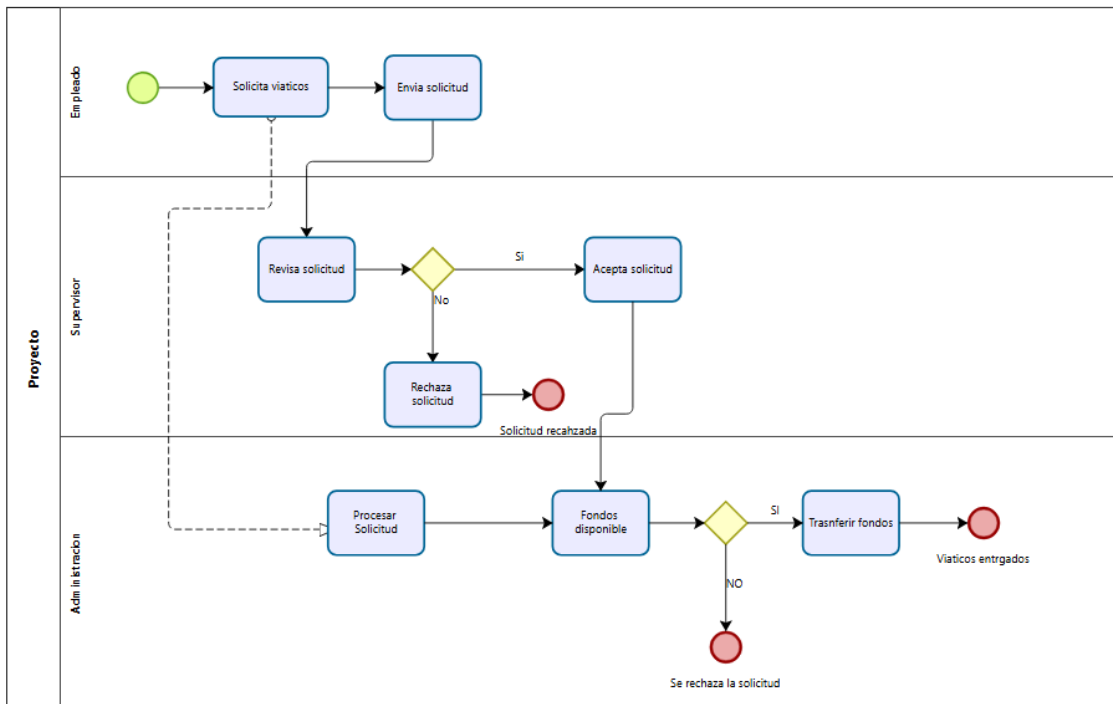
OBJETIVOS

Deberán identificar un proceso administrativo crítico dentro de una organización (ej. gestión de vacaciones, soporte técnico nivel 1, alta de proveedores) y automatizarlo mediante un chatbot, asegurando que la lógica del bot responda a un modelo de procesos estandarizado. Realizarlo como Simulación de proceso

CONSIGNA

El trabajo práctico comienza con la simulación de una problemática, consta del accionar de una empresa para administrar la solicitud de viáticos para sus empleados en diferentes proyectos.

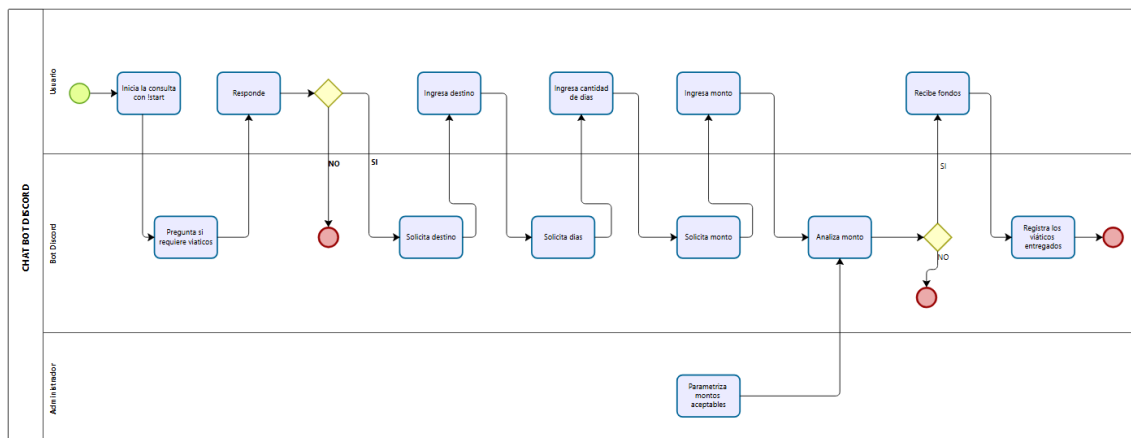
Se parte del análisis As-Is mediante el siguiente diagrama BPMN



Mediante el diagrama le exponemos a la empresa como en el modelo actual utiliza demasiado recursos, se parte de la necesidad del empleado de contar con viáticos, esta pasa directamente al supervisor del proyecto que da conformidad o no a la petición, y a su vez la administración debe controlar el presupuesto asignado a ese proyecto.

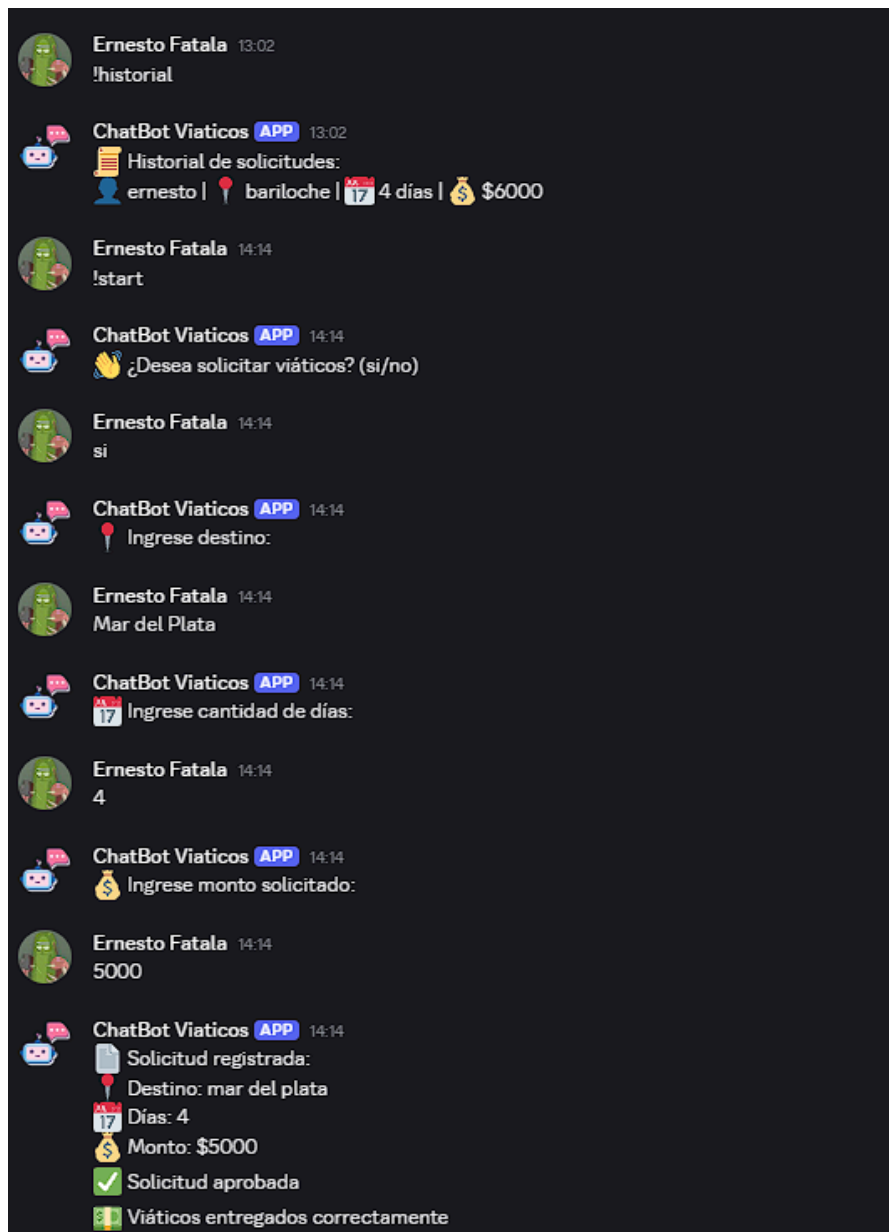
Luego de presentar la problemática, se le presenta a la empresa un proyecto de automatización con la implementación de un chat Bot.

Se le presenta el diagrama To-Be



En este flujo podemos presentarle a la empresa una solución automatizando la petición de viáticos con la responsabilidad de un solo administrador para parametrizar las opciones presentadas.

Se le presentan pantallas del Bot en su estado beta



Presentada la solución en su etapa Beta se espera la aprobación para adaptar el desarrollo a la necesidad de la empresa

ARQUITECTURA TECNICA DE LA SOLUCION

La solución se implementa como una simulación funcional de chatbot. La plataforma seleccionada es Discord, ya que permite representar una conversación real con el usuario y facilita la demostración del proceso en vivo durante la defensa.

Componente	Tecnologia	Funcion
Lenguaje	Python	Permite implementar la lógica conversacional y las reglas de decisión.
Plataforma	Discord	Canal utilizado para que el usuario interactúe con el bot.
Libreria	discord.py	Permite conectar el código Python con la API de Discord.
Persistencia	CSV	Archivo utilizado como base de datos simulada para registrar solicitudes.
Modelado	BPMN 2.0	Representa el proceso actual y el proceso automatizado.

El bot utiliza una máquina de estados para recordar en qué etapa del proceso se encuentra cada usuario. Los estados principales son: inicio, confirmación, destino, días, monto y fin.

REGLAS DE NEGOCIO

- El usuario debe iniciar el proceso con el comando !start.
- Si el usuario no confirma la solicitud, el proceso se cancela.
- La cantidad de días debe ser un número entero mayor a cero.
- El monto solicitado debe ser numérico.
- Si el monto es menor a \$100000, la solicitud se aprueba inicialmente.
- Si el monto es igual o mayor a \$100000, la solicitud se rechaza por monto elevado.
- Si existen fondos disponibles, el bot confirma la entrega de viáticos.
- Si no existen fondos disponibles, el bot informa la imposibilidad de completar la entrega.

DICCIONARIO DE DATOS

La base de datos simulada se encuentra en el archivo data/datos.csv. Cada fila representa una solicitud de viáticos registrada por el bot.

Campo	Tipo	Descripción	Ejemplo
Usuario	Texto	Nombre del usuario que realiza la solicitud.	ernesto
Destino	Texto	Lugar o ciudad a la que viaja el empleado.	bariloche
Días	Entero	Cantidad de días solicitados para el viaje.	4
Monto	Entero	Importe solicitado para cubrir viáticos.	6000

MANUAL DE USUARIO

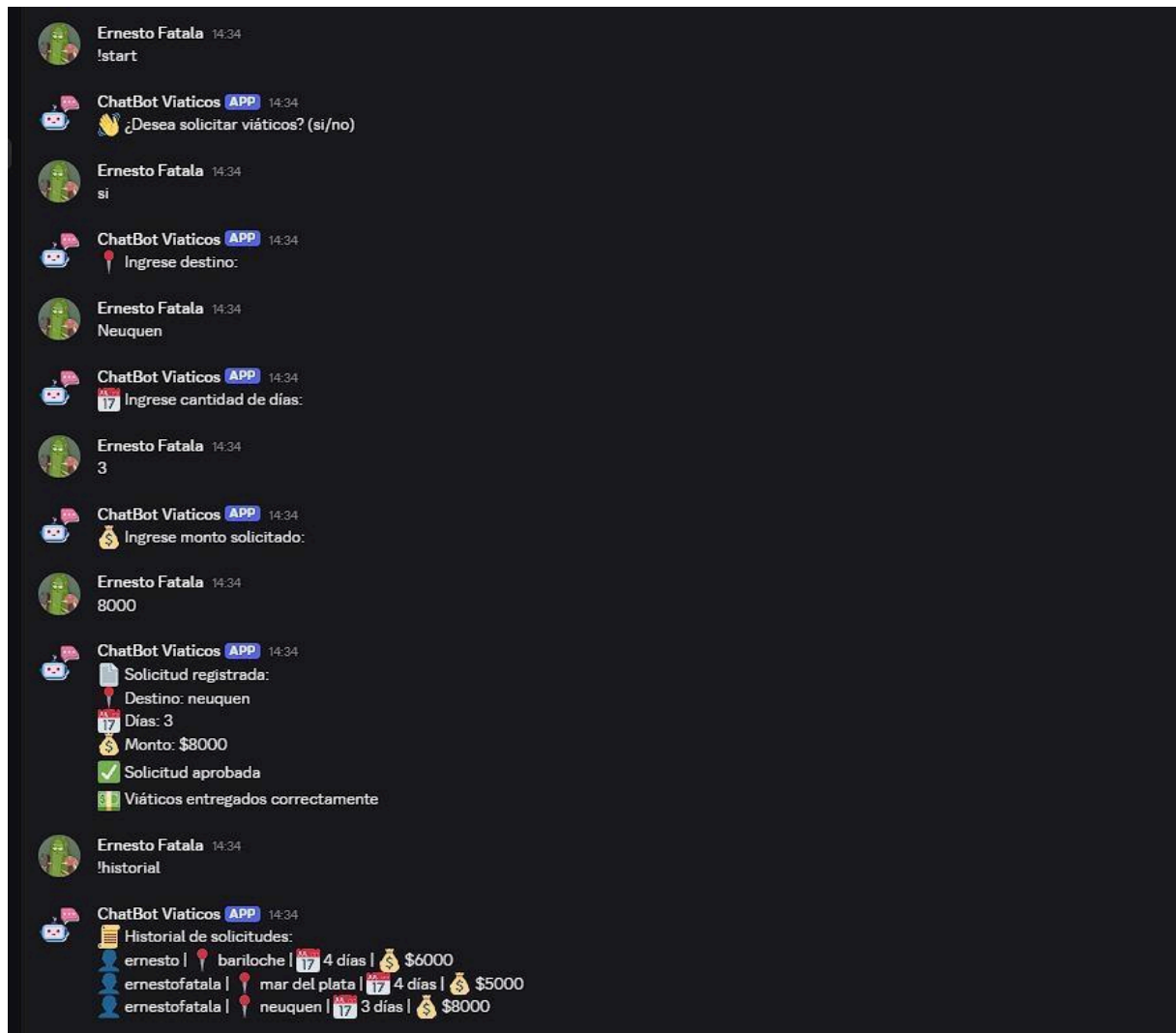
Este manual describe el uso básico del bot desde Discord.

1. Ingresar al servidor de Discord donde se encuentra agregado el bot.
2. Escribir el comando !start para iniciar una nueva solicitud de viáticos.
3. Responder si cuando el bot pregunte si desea solicitar viáticos. Si responde no, el proceso se cancela.
4. Ingresar el destino solicitado cuando el bot lo pida.
5. Ingresar la cantidad de días. Debe ser un número válido mayor a cero.
6. Ingresar el monto solicitado. Debe ser un valor numérico.
7. Leer la respuesta final del bot, que puede aprobar la solicitud, rechazarla o informar falta de fondos.
8. Usar el comando !historial para consultar las últimas solicitudes registradas.

PRUEBAS Y MANEJO DE ERRORES

El sistema contempla caminos felices e infelices. Las pruebas permiten verificar que el bot responde correctamente ante datos válidos, datos inválidos y decisiones de negocio.

Caso	Entrada o situacion	Resultado esperado
Camino feliz	Destino, días y monto validos menor a \$100000.	Solicitud registrada y viáticos entregados.
Cancelación	El usuario responde no al inicio.	Proceso cancelado.
Dias invalidos	El usuario ingresa texto o cero en cantidad de días.	El bot pide un número válido.
Monto invalido	El usuario ingresa texto como monto.	El bot pide un monto válido.
Monto elevado	El usuario ingresa un monto igual o mayor a \$100000.	Solicitud rechazada por monto elevado.
Historial	El usuario ejecuta !historial.	El bot muestra las últimas solicitudes.



USO DE HERRAMIENTAS DE IA

Durante el desarrollo del trabajo se utilizaron las siguientes herramientas de inteligencia artificial:

NotebookLM como organizador de los pasos en la realización del TP.

ChatGPT como apoyo para organizar la documentación, revisar la coherencia entre BPMN y código, proponer mejoras en el README, diccionario de datos y manual de usuario y para consultar que opción de Bot conviene más para una primera experiencia programándolo

viaticos_to_be_discord.bpmn

index.py

Necesito que revises los siguientes BPMN y el siguiente código, que te voy a adjuntar, y me digas si son coherentes, si no son coherentes decime los puntos a modificar

2:18 p.m.  

Procesado durante 34s >

Sí: **son coherentes en lo general**. El BPMN TO-BE representa bastante bien el flujo real del bot: `!start`, confirmación, carga de destino, días, monto, validaciones, registro en CSV, evaluación de monto y fondos.

Pero hay algunos puntos que conviene corregir para que quede más sólido en la defensa.

Puntos A Modificar

1. La rama "No hay fondos" del BPMN hoy no puede ocurrir en el código

En [index.py \(line 151\)](#) el bot solo sigue si `monto < 100000`.

Después, en [index.py \(line 159\)](#) define:

```
python
fondos = 200000
```

Entonces, si el monto siempre es menor a `100000`, y los fondos son `200000`, la condición `fondos >= monto` siempre va a dar verdadera. La rama "No hay fondos disponibles" existe en el BPMN, pero en la práctica nunca se ejecuta.

Recomendación: cambiar los fondos simulados a un  menor, por ejemplo `fondos = 50000`, o hacer que los fondos vengan de un CSV/configuración.

Conclusión

El BPMN y el código **sí son coherentes a nivel de proceso**, pero yo corregiría sí o sí estos tres puntos antes de entregar:

1. Hacer posible la rama "No hay fondos".
2. Alinear "consulta fondos simulados" con lo que realmente hace el código.
3. Corregir caracteres rotos del `index.py`.

Con eso queda mucho más defendible.

USO DEL REPOSITORIO

Se trabajo en conjunto en el repositorio: https://github.com/Ernestoabel/TPI_OE.git se utilizaron las medidas de seguridad aprendidas en el TP anterior, mediante la creación de un token de seguridad.

CONCLUSIÓN

La consigna del trabajo práctico nos permitió abordar una problemática simple para aprender a analizar una situación y proponer una solución orientada a optimizar procesos. En este sentido, el modelado BPMN resultó clave, puesto que su formato visual facilita la rápida identificación de las tareas a mejorar. Asimismo, la incorporación de herramientas de inteligencia artificial aportó un valor significativo; no obstante, tal como se mencionó en el informe anterior, su uso debe limitarse a consultas en etapas específicas. De lo contrario, delegar la resolución completa en la IA puede ser contraproducente, debido a su tendencia a generar respuestas con errores y problemas de continuidad.

La presentación de la solución y su implementación mediante el chatbot constituyeron una experiencia de aprendizaje dinámica y necesaria, la cual evidenció el potencial de mejora continua del sistema. Por este motivo, la propuesta se presenta en una fase beta o preliminar, proyectándose como una base sólida para su eventual validación y ejecución como un proyecto real.